

RFID Runtime Playbook: 11 Examples

This document maps runtime log patterns to the exact code paths in `app/src/main/java/com/zebra/rfid/demo/sdksample/MainActivity.java` and `app/src/main/java/com/zebra/rfid/demo/sdksample/RFIDHandler.java`.

How to use this file: - Match your log sequence to one example. - Verify source anchors first. - Compare expected status strings. - Use thread context to debug timing issues.

Eleven Examples

1. Example One: Init and Connect
2. Example Two: Background and Disconnect
3. Example Three: Foreground and Reconnect
4. Example Four: Battery Remove and Disconnect
5. Example Five: Battery Insert and Reconnect
6. Example Six: RFD40 Sled Detach and Disconnect
7. Example Seven: RFD40 Sled Attach and Reconnect
8. Example Eight: USB Cable Connected -> Reader Disappear and Disconnect
9. Example Nine: USB Cable Unplug and Reconnect Window
10. Example Ten: Power Connected -> Disconnected -> Reconnect
11. Example Eleven: Power Disconnected While Already Connected

1) Example One: Init and Connect

Thread context

- Main thread: activity lifecycle (`onCreate`, `onPostResume`)
- Background executor: SDK init (`createInstanceAndConnect`) and connection task
- Scheduled executor: deferred connect (`finishConnectionAttempt(connect())`)

Source anchors

- `MainActivity.java#L522 initializeRfidHandlerIfPermitted`
- `MainActivity.java#L492 onPostResume`
- `RFIDHandler.java#L64 onCreate`
- `RFIDHandler.java#L91 onResume`
- `RFIDHandler.java#L125 initSdk`
- `RFIDHandler.java#L181 connectReader`
- `RFIDHandler.java#L333 selectReader`
- `RFIDHandler.java#L427 connect`

Expected status strings

- Initializing reader... or Preparing reader...
- Connecting...
- Connected: `<reader-hostname> (<ms> ms)`

Representative snippet

```
// MainActivity -> RFIDHandler bootstrap
private void initializeRfidHandlerIfPermitted() {
    if (!canUseRfidHandler() || rfidHandlerInitialized) {
        return;
    }
    Log.d(TAG, "Step 2: rfidHandler.onCreate for this MainActivity with UI Response Interface");
}
```

```

        rfidHandler.onCreate(this, this);
        rfidHandlerInitialized = true;
    }

    @Override
    protected void onResume() {
        super.onResume();
        Log.d(TAG, "STEP: onResume requestReaderResumeDebounce");
        requestReaderResumeDebounce("activity_post_resume", false);
    }

    // RFIDHandler connect pipeline
    private void initSdk() {
        Log.d(TAG, "STEP: InitSDK");
        if (readers == null) {
            if (initializationInProgress) {
                Log.d(TAG, "STEP: Skip Duplicated InitSDK");
                return;
            }
            Log.d(TAG, "STEP: InitSDK initializationInProgress, createInstanceAndConnect");
            initializationInProgress = true;
            runOnBackground(this::createInstanceAndConnect);
        } else if (resumeRequested) {
            connectReader();
        }
    }
}

```

2) Example Two: Background and Disconnect

Thread context

- Main thread: activity pause callback
- Background executor: beep tone operation

Source anchors

- MainActivity.java#L482 onPause
- RFIDHandler.java#L110 onPause
- RFIDHandler.java#L498 disconnect

Expected status strings

- Detach path: Reader detached: <name>
- Manual/background path: Disconnected

Representative snippet

```

// MainActivity
@Override
protected void onPause() {
    super.onPause();
    stopPowerReconnectWindow();
    if (rfidHandlerInitialized) {
        Log.d(TAG, "STEP: onPause, disconnect");
        rfidHandler.onPause();
    }
}

```

```

    }
}

// RFIDHandler
void onPause() {
    Log.d(TAG, "STEP: RFIDHandler onPause: disconnecting");
    resumeRequested = false;
    connectionInProgress = false;
    disconnect();
}

```

3) Example Three: Foreground and Reconnect

Thread context

- Main thread: resume debounce and status update
- Background executor: connection task inside `connectReader`

Source anchors

- MainActivity.java#L741 `requestReaderResumeDebounced`
- MainActivity.java#L697 `requestReaderResume`
- RFIDHandler.java#L91 `onResume`

Expected status strings

- Connecting...
- Connected: <reader-hostname> (<ms> ms)

Representative snippet

```

// MainActivity
private void requestReaderResumeDebounced(String source, boolean force) {
    long now = SystemClock.elapsedRealtime();
    if (!force && now - lastReaderResumeRequestAtMs < READER_RESUME_DEBOUNCE_MS) {
        Log.d(TAGUSB, "Skipping reconnect request due to debounce window. source=" + source);
        return;
    }

    lastReaderResumeRequestAtMs = now;
    requestReaderResume();
}

private void requestReaderResume() {
    if (!rfidHandlerInitialized) {
        initializeRfidHandlerIfPermitted();
    }

    if (!rfidHandlerInitialized) {
        applyReaderStatus(getString(R.string.status_permission_required));
        return;
    }

    String result = rfidHandler.onResume();
    if (result != null && !result.isEmpty() && !shouldSuppressReconnectStatus(result)) {

```

```

        applyReaderStatus(result);
    }
}

```

4) Example Four: Battery Remove and Disconnect

Thread context

- SDK callback thread: `RFIDReaderDisappeared`
- Background executor: beep tone

Source anchors

- `RFIDHandler.java#L420` `RFIDReaderDisappeared`
- `RFIDHandler.java#L498` `disconnect`

Expected status strings

- Reader detached: `<name>`
- Disconnected may be suppressed for detach events

Representative snippet

```

@Override
public void RFIDReaderDisappeared(ReaderDevice readerDevice) {
    Log.d(TAG, "RFIDReaderDisappeared " + readerDevice.getName());
    detachedEventInProgress = true;
    responseHandler.sendToast("Reader detached: " + readerDevice.getName());
    disconnect();
}

```

5) Example Five: Battery Insert and Reconnect

Thread context

- SDK callback thread: `RFIDReaderAppeared`
- Scheduled executor: deferred `connectReader`

Source anchors

- `RFIDHandler.java#L412` `RFIDReaderAppeared`
- `RFIDHandler.java#L690` `beepAppear`
- `RFIDHandler.java#L181` `connectReader`

Expected status strings

- Reader attached: `<name>`
- Connecting...
- Connected: `<reader-hostname> (<ms> ms)`

Representative snippet

```

@Override
public void RFIDReaderAppeared(ReaderDevice readerDevice) {
    Log.d(TAG, "RFIDReaderAppeared " + readerDevice.getName());
    responseHandler.sendToast("Reader attached: " + readerDevice.getName());
    beepAppear();
}

```

```

        scheduleOnBackground(this::connectReader, READER_APPEAR_DELAY_MS);
    }

```

6) Example Six: RFD40 Sled Detach and Disconnect

Thread context

- Same as Example Four (shared callback path)

Source anchors

- RFIDHandler.java#L420 RFIDReaderDisappeared
- RFIDHandler.java#L498 disconnect

Expected status strings

- Reader detached: <name>

Representative snippet

```

// Shared with Example Four
@Override
public void RFIDReaderDisappeared(ReaderDevice readerDevice) {
    Log.d(TAG, "RFIDReaderDisappeared " + readerDevice.getName());
    detachedEventInProgress = true;
    responseHandler.sendToast("Reader detached: " + readerDevice.getName());
    disconnect();
}

```

7) Example Seven: RFD40 Sled Attach and Reconnect

Thread context

- Same as Example Five (shared callback path)

Source anchors

- RFIDHandler.java#L412 RFIDReaderAppeared
- RFIDHandler.java#L181 connectReader

Expected status strings

- Reader attached: <name>
- Connected: <reader-hostname> (<ms> ms)

Representative snippet

```

// Shared with Example Five
@Override
public void RFIDReaderAppeared(ReaderDevice readerDevice) {
    Log.d(TAG, "RFIDReaderAppeared " + readerDevice.getName());
    responseHandler.sendToast("Reader attached: " + readerDevice.getName());
    beepAppear();
    scheduleOnBackground(this::connectReader, READER_APPEAR_DELAY_MS);
}

```

8) Example Eight: USB Cable Connected -> Reader Disappear and Disconnect

Thread context

- Main thread: USB broadcast (ACTION_POWER_CONNECTED log and USB-sharing modes check)
- SDK callback thread: detach event

Source anchors

- MainActivity.java#L247 mBatteryReceiver
- RFIDHandler.java#L420 RFIDReaderDisappeared

Expected status strings

- USB log: USB Client action: android.intent.action.ACTION_POWER_CONNECTED
- If power-only mode: RFID disconnected while USB power cable connected or Disconnected
- If USB file transfer: USB file transfer active or debug mode\r\nRFID Disconnected\r\nWait for USB cable unplug for reconnect

Representative snippet

```
if (Intent.ACTION_POWER_CONNECTED.equals(action)) {
    refreshUsbModeFromStickyState();

    if (powerConnectedLatched) {
        Log.d(TAGUSB, "STEP: Skip ACTION_POWER_CONNECTED because latch is active (waiting for disconnect)");
        return;
    }

    long now = SystemClock.elapsedRealtime();
    if (now - lastPowerConnectedHandledAtMs < POWER_CONNECTED_EVENT_DEBOUNCE_MS) {
        Log.d(TAGUSB, "STEP: Skip duplicate ACTION_POWER_CONNECTED within debounce window");
        return;
    }
    lastPowerConnectedHandledAtMs = now;
    powerConnectedLatched = true;

    Log.d(TAGUSB, "STEP: ACTION_POWER_CONNECTED");
    Log.d(TAGUSB, "ACTION_POWER_CONNECTED usbState: connected=" + usbConnected
        + ", configured=" + usbConfigured
        + ", fileTransfer=" + usbFileTransferModeActive);
    boolean handlerBusy = rfidHandlerInitialized && rfidHandler != null && rfidHandler.isConnectionBusy;
    Log.d(TAGUSB, "ACTION_POWER_CONNECTED state: reconnectWindowActive=" + powerReconnectWindowActive + ", handlerBusy=" + handlerBusy);

    if (powerReconnectWindowActive) {
        Log.d(TAGUSB, "STEP: ACTION_POWER_CONNECTED during reconnect window -> cancel reconnect flow");
    }
    stopPowerReconnectWindow();
    suppressReconnectStatusUntilConnected = false;

    if (!rfidHandlerInitialized || rfidHandler == null) {
        return;
    }

    if (rfidHandler.isTC22R()) {
        Log.d(TAGUSB, "STEP: TC22R, ignore ACTION_POWER_CONNECTED");
    }
}
```

```

        onReaderStatusUpdate("Connected to TC22R");
        return;
    }

    // File transfer mode policy: keep RFID session until actual reader disappearance.
    if (usbFileTransferModeActive) {
        rfidHandler.setSkipTc53eReaderSelection(false);
        Log.d(TAGUSB, "STEP: ACTION_POWER_CONNECTED in file-transfer mode -> keep RFID connected until :");
        sendToast("USB file transfer active or debug mode\r\nRFID Disconnected\r\nWait for USB cable unplugged");
        return;
    }

    // Power-only cable policy: stop current RFID session and avoid selecting host RFIDTC53E while power is on
    rfidHandler.setSkipTc53eReaderSelection(true);
    Log.d(TAGUSB, "STEP: ACTION_POWER_CONNECTED power-only -> disconnect RFID");
    rfidHandler.onPause();
    sendToast("RFID disconnected while USB power cable connected");
}

```

9) Example Nine: USB Cable Unplug and Reconnect Window

Thread context

- Main thread: USB broadcast + reconnect window orchestration
- Main handler scheduled tasks: retry attempts and timeout
- Background executor: RFID connection work in handler

Source anchors

- MainActivity.java#L247 mBatteryReceiver
- MainActivity.java#L752 startPowerReconnectWindow
- MainActivity.java#L133 powerReconnectAttemptRunnable
- MainActivity.java#L713 shouldSuppressReconnectStatus

Expected status strings

- Starting power-unplug reconnect window
- Power reconnect attempt <n>/3
- Power reconnect deferred: handler busy
- transient suppression logs until connected

Failure variant (important)

- First or early attempt may return `Connection failed: RFID_COMM_OPEN_ERROR`.
- This is expected during cable transition races and should recover via bounded retries.

Representative snippet

```

if (Intent.ACTION_POWER_DISCONNECTED.equals(action)) {
    Log.d(TAGUSB, "ACTION_POWER_DISCONNECTED");
    powerConnectedLatched = false;
    usbConnected = false;
    usbConfigured = false;
    usbFileTransferModeActive = false;
    if (rfidHandler == null)
        return;
}

```

```

        rfidHandler.setSkipTc53eReaderSelection(false);
        if (rfidHandler.isReaderConnected()){
            stopPowerReconnectWindow();
            Log.d(TAG, "Reader already connected");
            sendToast("USB unplugged. Reader Connected");
            return;
        }
        startPowerReconnectWindow();
    }
}

```

10) Example Ten: Power Connected -> Disconnected -> Reconnect

Thread context

- Main thread: power event sequence and reconnect suppression
- SDK callback thread: detach/attach callbacks during cable transitions
- Main handler scheduled tasks: retry loop while busy

Source anchors

- MainActivity.java#L247 mBatteryReceiver
- MainActivity.java#L752 startPowerReconnectWindow
- MainActivity.java#L133 powerReconnectAttemptRunnable
- RFIDHandler.java#L181 connectReader

Expected status strings

- USB Client action: android.intent.action.ACTION_POWER_CONNECTED
- ACTION_POWER_DISCONNECTED
- Power reconnect deferred: handler busy
- final Connected: ... status

Failure variant (important)

- During rapid cable transitions, reader selection can temporarily pick host reader (DEFAULT) before external reader reappears.
- This is acceptable if final status reaches connected and suppression clears.

Representative snippet

```

private final Runnable powerReconnectAttemptRunnable = new Runnable() {
    @Override
    public void run() {
        if (!powerReconnectWindowActive) {
            return;
        }

        if (rfidHandler == null || (rfidHandlerInitialized && rfidHandler.isReaderConnected())) {
            stopPowerReconnectWindow();
            return;
        }

        if (rfidHandlerInitialized && rfidHandler.isConnectionBusy()) {
            Log.d(TAGUSB, "Power reconnect deferred: handler busy");
            mainHandler.postDelayed(this, POWER_RECONNECT_BUSY_RETRY_DELAY_MS);
            return;
        }
    }
}

```



```

    }

    int attemptNumber = powerReconnectAttemptIndex + 1;
    Log.d(TAGUSB, "Power reconnect attempt " + attemptNumber + "/" + POWER_RECONNECT_ATTEMPT_DELAYS_MS.length);
    requestReaderResumeDebounce("power_disconnected_retry_" + attemptNumber, true);

    powerReconnectAttemptIndex++;
    if (powerReconnectAttemptIndex < POWER_RECONNECT_ATTEMPT_DELAYS_MS.length) {
        mainHandler.postDelayed(this, POWER_RECONNECT_ATTEMPT_DELAYS_MS[powerReconnectAttemptIndex]);
    }
}

};

```

Sequence diagram

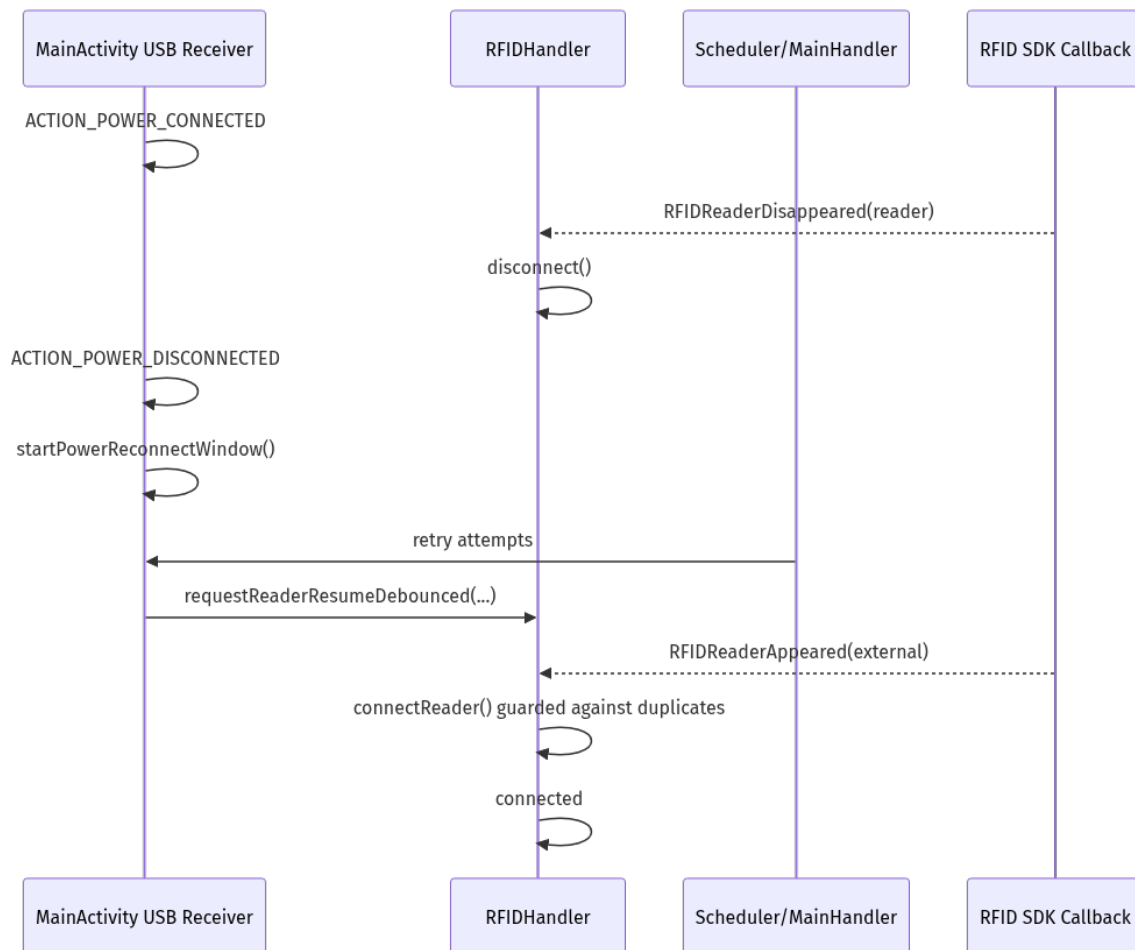


Figure 1: Example Ten Sequence Diagram

11) Example Eleven: Power Disconnected While Already Connected

Thread context

- Main thread only: receiver short-circuit path

Source anchors

- MainActivity.java#L247 mBatteryReceiver

Expected status strings

- ACTION_POWER_DISCONNECTED
- Reader already connected
- USB unplugged. Reader Connected

Representative snippet

```
if (Intent.ACTION_POWER_DISCONNECTED.equals(action)) {
    Log.d(TAGUSB, "ACTION_POWER_DISCONNECTED");
    powerConnectedLatched = false;
    usbConnected = false;
    usbConfigured = false;
    usbFileTransferModeActive = false;
    if (rfidHandler == null)
        return;
    rfidHandler.setSkipTc53eReaderSelection(false);
    if (rfidHandler.isReaderConnected()){
        stopPowerReconnectWindow();
        Log.d(TAG, "Reader already connected");
        sendToast("USB unplugged. Reader Connected");
        return;
    }
    startPowerReconnectWindow();
}
```

Sequence diagram

Suggestions for Ongoing Maintenance

- Keep source anchors current after refactors (`grep -n update pass`).
- Keep expected status strings synchronized with `onReaderStatusUpdate` and toast text.
- Preserve failure variants for cable-transition examples; they reduce false-positive bug reports.
- If new logs are added, include thread context first, then snippets.

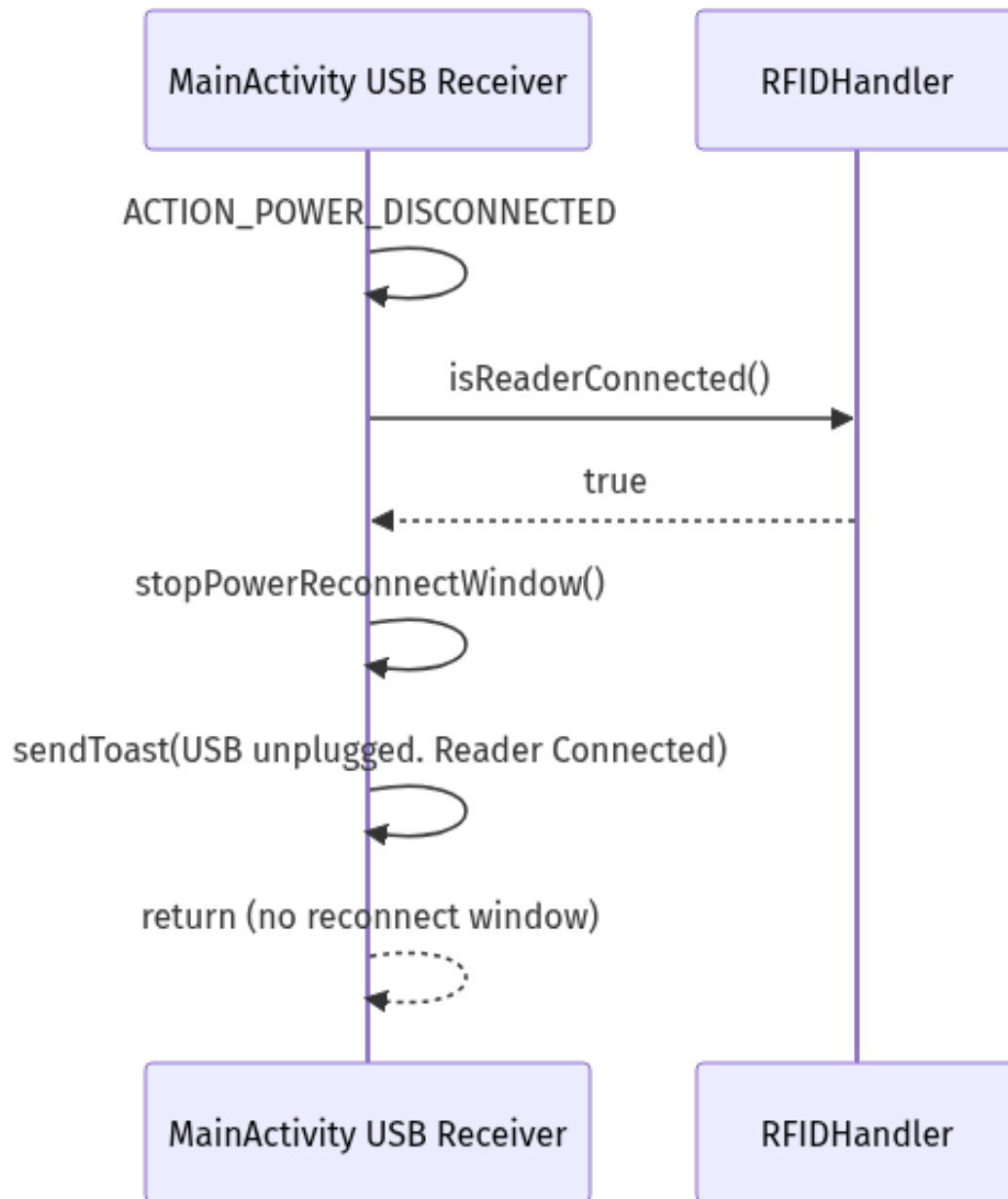


Figure 2: Example Eleven Sequence Diagram